

Design & Analysis of Algorithms

Lab 1: Algorithm Performance Analysis Workbench

Name: Priyam Sharma

Roll No: 2501730184

Email: 2501730184@krmu.edu.in

Q1. 1.5 Runtime and Complexity Comparison Table (5 marks)

```
def generate_runtime_complexity_table(n):
    import math

    return [
        "Runtime Complexity Comparison",
        "Method ObservedCount ExpectedComplexity Observation",
        f"Linear Search {n} O(n) Grows linearly",
        f"Binary Search {math.floor(math.log2(n)) + 1} O(log n) Grows logarithmically",
        f"Bubble Sort {n * (n - 1) // 2} O(n^2) Grows quadratically",
        f"Insertion Sort {n * (n - 1) // 2} O(n^2) Grows quadratically"
    ]
```

Q2. 1.6 Runtime Comparison Chart Data and Scalability Report (5 marks)

```
def generate_runtime_chart_report(input_sizes):
    import math

    result = []

    result.append("Runtime Comparison Chart Data")
    result.append("InputSize LinearSearch BinarySearch BubbleSort InsertionSort")

    for n in input_sizes:
        linear = n
        binary = math.floor(math.log2(n)) + 1
        bubble = n * (n - 1) // 2
        insertion = n * (n - 1) // 2

        result.append(f"{n} {linear} {binary} {bubble} {insertion}")

    result.append("Scalability Summary")
    result.append("Algorithm Complexity Scalability")
    result.append("Linear Search O(n) Moderate")
    result.append("Binary Search O(log n) Excellent")
    result.append("Bubble Sort O(n^2) Poor")
    result.append("Insertion Sort O(n^2) Poor")
    result.append("Key Observations")
    result.append("Best Algorithm: Binary Search")
    result.append("Most Expensive Algorithm: Bubble Sort")
    result.append("Conclusion: Logarithmic algorithms scale better for large inputs")

    return result
```